



Adopt-a-Dog

Flutter Application

Student Development Manual

Level	Platform
Beginner–Intermediate	Flutter (Android / iOS)

Topics Covered

Dog CEO API • FutureBuilder • Image.network • shared_preferences

Total Points: 50	
Core Implementation 35 points	Exercises (×4) 15 points

1. Introduction

Welcome to the Adopt-a-Dog Flutter project! This manual guides you through building a mobile app that fetches real dog images from the Dog CEO API, displays them using Flutter's FutureBuilder and Image widgets, and remembers a user's favorite dog breed across app sessions using shared_preferences.

By the end of this project you will be able to:

- Consume a public REST API inside a Flutter application.
- Use FutureBuilder to manage asynchronous state in the UI.
- Display remote images with the Image.network widget.
- Persist simple key-value data locally with the shared_preferences package.
- Understand the basic architecture of a single-feature Flutter app.

What You Will Build

A scrollable list of dog breeds fetched from the Dog CEO API.

Tap any breed to see a random image of that breed.

A heart button to save your favorite breed — it is remembered the next time you open the app.

A Favorites screen that shows the last saved breed and its image.

2. Prerequisites

Before you start, make sure you have the following installed and working on your development machine.

Tool	Minimum Version	Notes
Flutter SDK	3.x	Includes Dart 3
Dart	3.0	Bundled with Flutter
Android Studio / VS Code	Latest stable	Install Flutter & Dart plugins
Android Emulator / iOS Sim	API 21+ / iOS 12+	Or a physical device

Internet connection	Required	Fetches images at runtime
---------------------	----------	---------------------------

3. Project Setup

3.1 Create the Flutter Project

Open a terminal, navigate to your workspace folder, and run:

Terminal

```
flutter create adopt_a_dog
cd adopt_a_dog
```

3.2 Add Dependencies

Open pubspec.yaml and add the http and shared_preferences packages under the dependencies section:

pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0
  shared_preferences: ^2.2.3
```

Save the file, then fetch the packages:

Terminal

```
flutter pub get
```

3.3 Project File Structure

Your lib/ directory should be organised as follows. Create any missing files before writing code.

Project Structure

```
lib/
  main.dart          ← App entry point & MaterialApp
  models/
    breed.dart       ← Breed data model
  services/
```

```
dog_api_service.dart ← HTTP calls to Dog CEO API
prefs_service.dart   ← shared_preferences wrapper
screens/
  breed_list_screen.dart ← Home screen (FutureBuilder)
  breed_detail_screen.dart ← Random image per breed
  favorites_screen.dart   ← Shows saved favorite
```

4. The Dog CEO API

The Dog CEO API (<https://dog.ceo/dog-api/>) is a free, public REST API that serves dog breed names and random breed images. No API key is required.

Endpoint	Method	Description
/api/breeds/list/all	GET	Returns all breeds and sub-breeds as a JSON map.
/api/breed/{breed}/images/random	GET	Returns one random image URL for the given breed.

4.1 Sample JSON Response — Breed List

GET <https://dog.ceo/api/breeds/list/all>

```
{
  "message": {
    "affenpinscher": [],
    "australian": ["shepherd"],
    "bulldog": ["boston", "english", "french"],
    ...
  },
  "status": "success"
}
```

4.2 Sample JSON Response — Random Image

GET <https://dog.ceo/api/breed/bulldog/images/random>

```
{
  "message": "https://images.dog.ceo/breeds/bulldog-french/n02108915_4133.jpg",
  "status": "success"
}
```

5. Step-by-Step Implementation

5.1 Data Model — breed.dart

A simple immutable class to represent a single dog breed.

```
lib/models/breed.dart

class Breed {
  final String name;           // e.g. 'bulldog'
  final List<String> subBreeds; // e.g. ['french', 'english']

  const Breed({required this.name, required this.subBreeds});

  // Display name with optional sub-breed
  String displayName([String? sub]) =>
    sub != null ? '$sub $name' : name;
}
```

5.2 API Service — dog_api_service.dart

All network calls are isolated in a single service class. The two methods return Futures so they work neatly with FutureBuilder.

```
lib/services/dog_api_service.dart

import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/breed.dart';

class DogApiService {
  static const _base = 'https://dog.ceo/api';

  /// Returns every breed as a flat list.
  Future<List<Breed>> fetchBreeds() async {
    final uri = Uri.parse('$_base/breeds/list/all');
    final response = await http.get(uri);

    if (response.statusCode != 200) {
      throw Exception('Failed to load breeds');
    }

    final Map<String, dynamic> data = jsonDecode(response.body);
    final Map<String, dynamic> message = data['message'];

    return message.entries
```

```
.map((e) => Breed(
  name: e.key,
  subBreeds: List<String>.from(e.value as List),
))
.toList()
..sort((a, b) => a.name.compareTo(b.name));
}

/// Returns one random image URL for [breedName].
Future<String> fetchRandomImage(String breedName) async {
  final uri = Uri.parse('$_base/breed/$breedName/images/random');
  final response = await http.get(uri);

  if (response.statusCode != 200) {
    throw Exception('Failed to load image');
  }

  final Map<String, dynamic> data = jsonDecode(response.body);
  return data['message'] as String;
}
```

5.3 Preferences Service — prefs_service.dart

The PrefsService wraps shared_preferences so the rest of the app never imports that package directly. This keeps things easy to test and swap later.

```
lib/services/prefs_service.dart

import 'package:shared_preferences/shared_preferences.dart';

class PrefsService {
  static const _keyFavorite = 'favorite_breed';

  /// Saves [breedName] as the user's favorite.
  Future<void> saveFavorite(String breedName) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_keyFavorite, breedName);
  }

  /// Returns the saved favorite, or null if none has been set.
  Future<String?> loadFavorite() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_keyFavorite);
  }

  /// Clears the saved favorite.
  Future<void> clearFavorite() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove(_keyFavorite);
  }
}
```

How shared_preferences Works

SharedPreferences.getInstance() returns a Future — always await it.
Data is stored as a key/value pair on the device's local storage (NSUserDefaults on iOS, SharedPreferences XML on Android).
Supported value types: String, int, double, bool, and List<String>.
Data persists until the app is uninstalled or you call remove().

5.4 Breed List Screen — FutureBuilder

This is the home screen. It uses FutureBuilder to show a loading spinner while breeds are downloading, an error message if the request fails, and a scrollable list on success.

lib/screens/breed_list_screen.dart

```
import 'package:flutter/material.dart';
import '../models/breed.dart';
import '../services/dog_api_service.dart';
import 'breed_detail_screen.dart';
import 'favorites_screen.dart';

class BreedListScreen extends StatefulWidget {
  const BreedListScreen({super.key});
  @override
  State<BreedListScreen> createState() => _BreedListScreenState();
}

class _BreedListScreenState extends State<BreedListScreen> {
  // Store the Future once so rebuilds don't re-fetch.
  late final Future<List<Breed>> _breedsFuture;
  final _service = DogApiService();

  @override
  void initState() {
    super.initState();
    _breedsFuture = _service.fetchBreeds();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Choose a Breed'),
        actions: [
          IconButton(
            icon: const Icon(Icons.favorite),
            onPressed: () => Navigator.push(
              context,
              MaterialPageRoute(
                builder: (_) => const FavoritesScreen(),
              ),
            ),
          ),
        ],
      ),
      body: FutureBuilder<List<Breed>>(
        future: _breedsFuture,
        builder: (context, snapshot) {
          // ① Still waiting
          if (snapshot.connectionState != ConnectionState.done) {
            return const Center(child: CircularProgressIndicator());
          }
        }
      )
    );
  }
}
```

```
// ② Error
if (snapshot.hasError) {
  return Center(child: Text('Error: ${snapshot.error}'));
}
// ③ Data ready
final breeds = snapshot.data!;
return ListView.builder(
  itemCount: breeds.length,
  itemBuilder: (context, index) {
    final breed = breeds[index];
    return ListTile(
      leading: const Icon(Icons.pets),
      title: Text(breed.name.toUpperCase()[0]
        + breed.name.substring(1)),
      trailing: const Icon(Icons.chevron_right),
      onTap: () => Navigator.push(
        context,
        MaterialPageRoute(
          builder: (_) =>
            BreedDetailScreen(breed: breed),
        ),
      ),
    );
  },
);
```

FutureBuilder — The Three States

ConnectionState.waiting / active → Show a loading indicator.

snapshot.hasError → Display a human-friendly error message.

snapshot.hasData → Render your widgets with snapshot.data!.

Store the Future in a field (not in build()) to prevent redundant API calls on every rebuild.

5.5 Breed Detail Screen — Image.network

This screen fetches a random dog image for the selected breed and displays it with Image.network. It also lets the user save that breed as their favorite.

```
lib/screens/breed_detail_screen.dart
```

```
import 'package:flutter/material.dart';
import '../models/breed.dart';
import '../services/dog_api_service.dart';
import '../services/prefs_service.dart';

class BreedDetailScreen extends StatefulWidget {
  final Breed breed;
  const BreedDetailScreen({super.key, required this.breed});
  @override
  State<BreedDetailScreen> createState() => _BreedDetailScreenState();
}

class _BreedDetailScreenState extends State<BreedDetailScreen> {
  late Future<String> _imageFuture;
  final _api = DogApiService();
  final _prefs = PrefsService();
  bool _saved = false;

  @override
  void initState() {
    super.initState();
    _imageFuture = _api.fetchRandomImage(widget.breed.name);
  }

  void _refresh() =>
    setState(() {
      _imageFuture = _api.fetchRandomImage(widget.breed.name);
      _saved = false;
    });

  Future<void> _saveFavorite() async {
    await _prefs.saveFavorite(widget.breed.name);
    setState(() => _saved = true);
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('${widget.breed.name} saved!')),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(widget.breed.name)),
      body: FutureBuilder<String>(
        future: _imageFuture,
        builder: (context, snapshot) {
```

```

    if (snapshot.connectionState != ConnectionState.done) {
      return const Center(child: CircularProgressIndicator());
    }
    if (snapshot.hasError) {
      return Center(child: Text('${snapshot.error}'));
    }
    final url = snapshot.data!;
    return Column(
      children: [
        Expanded(
          child: Image.network(
            url,
            fit: BoxFit.cover,
            width: double.infinity,
            loadingBuilder: (ctx, child, progress) =>
              progress == null
                ? child
                : const Center(
                    child: CircularProgressIndicator(),
                    errorBuilder: (ctx, err, stack) =>
                      const Icon(Icons.broken_image, size: 64),
                  ),
          ),
        Padding(
          padding: const EdgeInsets.all(16),
          child: Row(
            mainAxisAlignment: MainAxisAlignment.spaceEvenly,
            children: [
              ElevatedButton.icon(
                onPressed: _refresh,
                icon: const Icon(Icons.refresh),
                label: const Text('New Photo'),
              ),
              ElevatedButton.icon(
                onPressed: _saved ? null : _saveFavorite,
                icon: Icon(_saved
                  ? Icons.favorite
                  : Icons.favorite_border),
                label: const Text('Favorite'),
              ),
            ],
          ),
        ),
      ],
    );
  },
);
}

```

5.6 Favorites Screen

On launch this screen reads the saved breed from `shared_preferences`, then uses another `FutureBuilder` to load a fresh image of that breed.

```
lib/screens/favorites_screen.dart

import 'package:flutter/material.dart';
import '../services/dog_api_service.dart';
import '../services/prefs_service.dart';

class FavoritesScreen extends StatelessWidget {
  const FavoritesScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final prefs = PrefsService();
    final api = DogApiService();

    return Scaffold(
      appBar: AppBar(title: const Text('My Favorite')),
      body: FutureBuilder<String?>(
        future: prefs.loadFavorite(),
        builder: (context, prefSnap) {
          if (prefSnap.connectionState != ConnectionState.done) {
            return const Center(child: CircularProgressIndicator());
          }
          final breed = prefSnap.data;
          if (breed == null) {
            return const Center(
              child: Text('No favorite saved yet!\n'
                'Tap ♥ on any breed photo.'),
            );
          }
          // Nested FutureBuilder for the image
          return FutureBuilder<String>(
            future: api.fetchRandomImage(breed),
            builder: (context, imgSnap) {
              if (imgSnap.connectionState != ConnectionState.done) {
                return const Center(
                  child: CircularProgressIndicator();
                );
              }
              if (imgSnap.hasError) {
                return Center(child: Text('${imgSnap.error}'));
              }
              return Column(
                children: [
                  Expanded(
                    child: Image.network(
                      imgSnap.data!,
                      fit: BoxFit.cover,
                      width: double.infinity,
                    ),
                  ),
                  Padding(
                    padding: const EdgeInsets.all(16),
                    child: Text(
```

```
        'Your favorite: $breed',  
        style: Theme.of(context).textTheme.titleLarge,  
      ),  
    ),  
  ],  
);  
},  
);  
},  
),  
);  
},  
);  
}
```

5.7 App Entry Point — main.dart

```
lib/main.dart

import 'package:flutter/material.dart';
import 'screens/breed_list_screen.dart';

void main() => runApp(const AdoptADogApp());

class AdoptADogApp extends StatelessWidget {
  const AdoptADogApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Adopt-a-Dog',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorSchemeSeed: Colors.orange,
        useMaterial3: true,
      ),
      home: const BreedListScreen(),
    );
  }
}
```

6. Key Concepts Deep-Dive

6.1 FutureBuilder

`FutureBuilder<T>` is a built-in Flutter widget that rebuilds whenever the state of a `Future` changes. Its builder callback receives an `AsyncSnapshot<T>` with three useful properties:

Property / State	Meaning	What to show
<code>connectionState != done</code>	Future is pending	<code>CircularProgressIndicator</code>
<code>snapshot.hasError</code>	Future threw an exception	Error message text
<code>snapshot.hasData</code>	Future completed with data	Your content widget(s)

6.2 Image.network

`Image.network` fetches and caches an image from a URL. The two most important optional parameters for a polished user experience are:

- `loadingBuilder` — called repeatedly while the image downloads; use it to show a progress indicator.
- `errorBuilder` — called if the download fails; use it to show a fallback icon.

Use `fit: BoxFit.cover` with `Expanded` to fill the available space without distorting the image.

6.3 shared_preferences

`shared_preferences` is a Flutter plugin that stores simple data on the device. Here are the rules to remember:

- Always await `SharedPreferences.getInstance()` — it is asynchronous.
- Use typed getters/setters: `getString`, `setString`, `getBool`, `setInt`, etc.
- Keys are plain strings — define them as constants to avoid typos.
- Data survives app restarts but is deleted when the app is uninstalled.
- Do NOT store sensitive data (passwords, tokens) in `shared_preferences` — use `flutter_secure_storage` instead.

7. Common Errors & Fixes

Error / Symptom	Cause	Fix
SocketException / No internet	Emulator has no network or device is offline.	Toggle airplane mode, restart emulator, or check Wi-Fi.
setState() called after dispose()	_imageFuture callback fires after leaving the screen.	Guard with if (mounted) before setState().
Image stays blank / broken icon	URL is null or the API returned a non-image.	Check snapshot.data before passing to Image.network.
Favorite not persisting	Forgot to await prefs.setString.	Always await every SharedPreferences write call.
FutureBuilder rebuilds endlessly	Future created inside build(), causing a new Future each frame.	Move the Future to initState() and store it in a field.
Package not found	pubspec.yaml edited but flutter pub get not run.	Run flutter pub get in the terminal after editing dependencies.

8. Exercises

Complete the following exercises to reinforce what you have learned. Each exercise carries a point value that contributes toward the 50-point total (see Section 9).

Ex 1 — Sub-Breed
3 pts

Ex 2 — Multi-Favorites
4 pts

Ex 3 — Caching
4 pts

Ex 4 — Search & Filter
4 pts

Exercise 1 ★ — Sub-Breed Support

3 points

The Breed model already stores subBreeds. Modify BreedDetailScreen so that, when a breed has sub-breeds, a horizontal chip row appears above the image. Tapping a chip updates _imageFuture to use the full path (e.g. bulldog/french).

Scoring Rubric:

- Chip row renders only for breeds that have sub-breeds **[1 pt]**
- Tapping a chip updates the displayed image to that sub-breed **[1 pt]**
- Parent breed image loads correctly when no chip is selected **[1 pt]**

Exercise 2 ★ ★ — Multiple Favorites

4 points

Change PrefsService to store a List<String> of favorites instead of a single string. Update FavoritesScreen to show a ListView of all saved breeds. Add a delete icon next to each entry that removes it from the list.

Scoring Rubric:

- PrefsService correctly stores and retrieves a List<String> [1 pt]
- FavoritesScreen shows all saved breeds in a scrollable list [1 pt]
- Delete icon removes the correct breed and refreshes the list [1 pt]
- List persists across app restarts (no data lost) [1 pt]

Exercise 3 ★ ★ — Offline Caching**4 points**

Add the cached_network_image dependency to pubspec.yaml. Replace every Image.network call with CachedNetworkImage. Test that previously loaded images still appear when the device is set to airplane mode.

Scoring Rubric:

- cached_network_image added correctly to pubspec.yaml [1 pt]
- All Image.network calls replaced with CachedNetworkImage [1 pt]
- Placeholder spinner shown while image loads [1 pt]
- Cached images display correctly when device is offline [1 pt]

Exercise 4 ★ ★ ★ — Search & Filter**4 points**

Add a search bar to BreedListScreen. As the user types, filter the breed list client-side (no extra API calls). Persist the last search term in shared_preferences and pre-fill the search bar on the next app launch.

Scoring Rubric:

- Search bar is visible and functional in BreedListScreen [1 pt]
- Breed list filters correctly as the user types (case-insensitive) [1 pt]
- No additional API calls are triggered during filtering [1 pt]
- Last search term persisted and pre-filled on next app launch [1 pt]

9. Grading Rubric & Submission Checklist

The project is graded out of 50 points total across two categories: Core Implementation (30 pts) and Exercises (20 pts). Review every item before submitting.

Category	Max Points	Score
Project Setup & Dependencies	5	_____
API Service (DogApiService)	6	_____
FutureBuilder Implementation	6	_____
Image.network & loadingBuilder	5	_____
shared_preferences (PrefsService)	5	_____
Favorites Screen	3	_____
Code Quality & No Analysis Errors	3	_____
App Runs Without Crashes	2	_____
✦ Core Implementation Subtotal	35	_____
Exercise 1 — Sub-Breed Support	3	_____
Exercise 2 — Multiple Favorites	4	_____
Exercise 3 — Offline Caching	4	_____
Exercise 4 — Search & Filter	4	_____
✦ Exercises Subtotal	15	_____
★ TOTAL	50	/ 50

9.1 Submission Checklist

Tick every box before uploading your project. Items marked with a point value are graded.

☐	Requirement	Points
☐	pubspec.yaml includes http and shared_preferences with correct versions.	5 pts
☐	DogApiService fetches the breed list and throws on non-200 status codes.	3 pts
☐	DogApiService fetches a random image URL for a given breed name.	3 pts

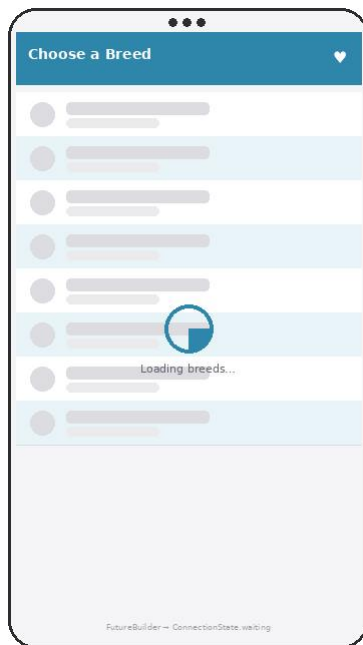
☐	BreedListScreen uses FutureBuilder and shows loading / error / data states.	6 pts
☐	BreedDetailScreen uses Image.network with loadingBuilder and errorBuilder.	5 pts
☐	Tapping Favorite calls PrefsService.saveFavorite and shows a SnackBar.	3 pts
☐	FavoritesScreen reads the stored breed on launch and displays its image.	3 pts
☐	Saved favorite persists after hot-restart (shared_preferences working).	2 pts
☐	No FutureBuilder creates its Future inside build() — Future stored in a field.	2 pts
☐	Code has no analysis warnings (dart analyze) and is formatted (dart format .).	3 pts
☐	Exercise 1 complete — sub-breed chip row working in BreedDetailScreen.	3 pts
☐	Exercise 2 complete — multiple favorites stored as List<String>.	4 pts
☐	Exercise 3 complete — CachedNetworkImage replaces Image.network.	4 pts
☐	Exercise 4 complete — search bar filters list; term persisted in prefs.	4 pts
☐	App runs on target device/emulator without crashes.	Pass/Fail
★	TOTAL SCORE	/ 50

10. App Wireframes

The wireframes below illustrate the expected layout for each screen and state. Use these as a visual reference while building your app. Exact styling may differ as long as the required widgets and behaviours are present.

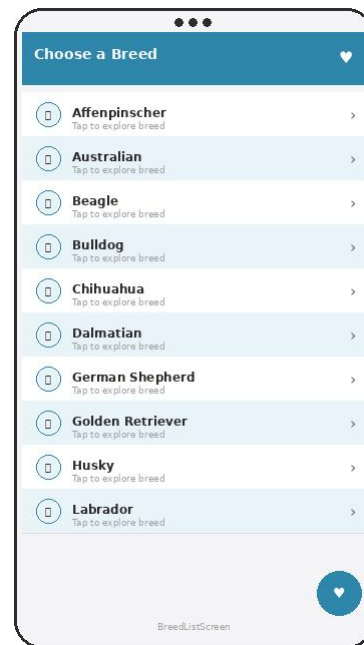
10.1 BreedListScreen

The home screen renders inside a FutureBuilder. The left wireframe shows the loading state (ConnectionState.waiting); the right shows the populated breed list.



Loading state

FutureBuilder → CircularProgressIndicator



Data state

FutureBuilder → ListView.builder

10.2 BreedDetailScreen

Reached by tapping any breed. The left wireframe shows Image.network's loadingBuilder during download; the right shows a fully loaded image with the action buttons.

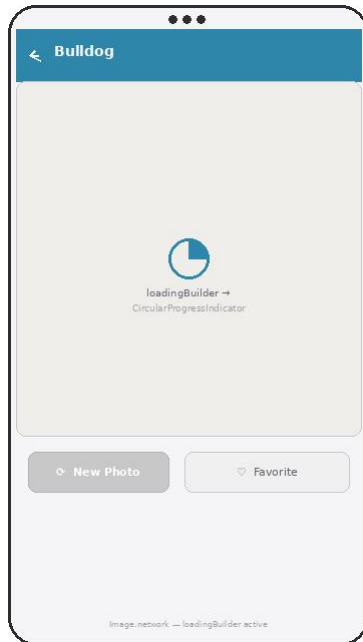


Image loading

loadingBuilder → spinner overlay

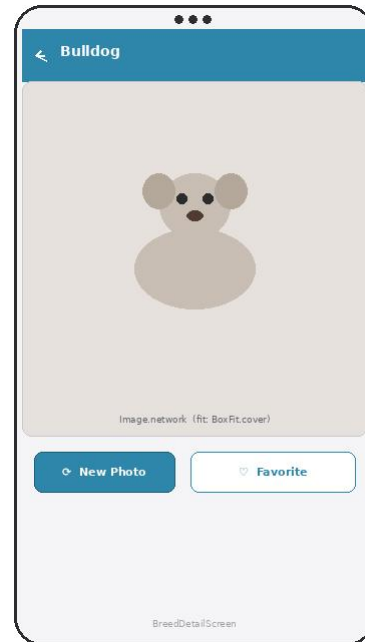
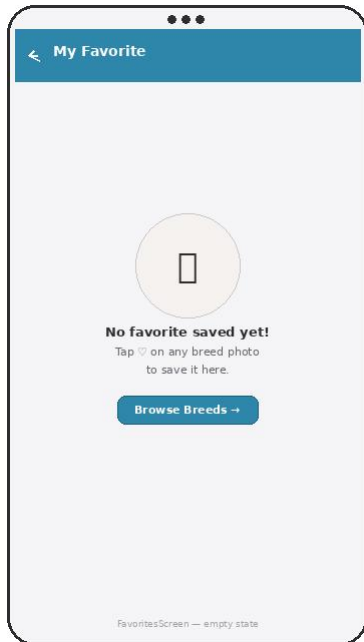


Image loaded

New Photo + Favorite buttons visible

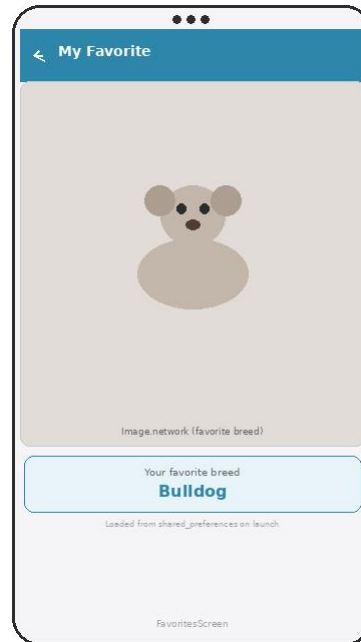
10.3 FavoritesScreen

Opened via the heart icon in the AppBar. The left wireframe shows the empty state when no breed has been saved yet; the right shows the screen after a breed was favorited.



Empty state

`prefs.loadFavorite()` returns null



Favorite loaded

Breed name + image from `shared_preferences`

11. Quick Reference

Resource	Link / Command
Dog CEO API Docs	https://dog.ceo/dog-api/documentation/
http package	pub.dev/packages/http
shared_preferences package	pub.dev/packages/shared_preferences
FutureBuilder docs	api.flutter.dev/flutter/widgets/FutureBuilder-class.html
Image.network docs	api.flutter.dev/flutter/widgets/Image/Image.network.html
Run the app	<code>flutter run</code>
Analyze code	<code>dart analyze</code>
Format code	<code>dart format .</code>

Happy coding — and happy dog-hunting!